

LANGSMITH CORE CONCEPTS CHEATSHEET

AI Engineer Ready · Week 6 — RAG Optimization + Evaluation + LangSmith

Golden Rule -> Project -> Trace -> Run: trace everything, debug in seconds instead of guessing blind

1. What Is LLM Observability?

	Detail
Trace every call	See the exact prompt, retrieved chunks, and output for every single request.
Debug fast	Find the failing step in seconds instead of guessing and re-running blind.
Prove quality over time	Track scores and latency across versions, not just at launch.

Observability is what evaluation looks like once your bot is live and talking to real users, not just your test set. "It's not working" is not a bug report -- a trace that shows the exact prompt, the retrieved chunks, and the raw model output is.

2. Core Concepts: Project -> Trace -> Run

Term	Meaning
Project	Your whole app, e.g. "Insurance Support Bot".
Trace	One full user request, start to finish.
Run	One step inside that trace -- a retrieval call, an LLM call, a tool call.

A useful analogy: a Project is a restaurant. A Trace is one customer's visit, walking in to walking out. A Run is one single action within that visit -- get seated, place the order, food comes out. Several Runs happen inside every Trace.

3. Two Ways to Get Traces

Method	How It Works	Use When
Automatic (LangChain)	One environment variable turns on tracing -- every chain and retriever call gets captured for free.	You're already building with LangChain / LangGraph.
Manual (@traceable)	Wrap your own custom Python functions with the @traceable decorator to log inputs, outputs, and timing.	Custom glue code outside LangChain that you still want visibility into.

```
$ # Automatic -- set these once, before running your LangChain app
$ export LANGCHAIN_TRACING_V2=true
$ export LANGCHAIN_API_KEY="your-langsmith-api-key"
$ export LANGCHAIN_PROJECT="Insurance Support Bot"

$ # Manual -- for your own custom functions
$ from langsmith import traceable

$ @traceable
$ def rerank_chunks(question, chunks):
$ ...
$ return sorted_chunks
```

LANGSMITH CORE CONCEPTS CHEATSHEET

AI Engineer Ready · Week 6 — RAG Optimization + Evaluation + LangSmith

Golden Rule -> Project -> Trace -> Run: trace everything, debug in seconds instead of guessing blind

You don't have to choose one -- most real projects mix both: automatic tracing for the LangChain parts, `@traceable` for the custom glue code around it.

4. Datasets, Evaluators & Feedback

Feature	What It Does
Datasets	A saved set of test questions you re-run every time you change something.
Custom evaluators	Your own faithfulness/relevance scoring functions, run automatically on every trace.
Feedback scoring	Attach a thumbs-up/down or a numeric score directly onto a specific run.

A dataset plus an evaluator turns "did I break anything" from a guess into a one-click re-run before every deploy.

5. Compare Runs & the Prompt Hub

Feature	What It Does
Compare Runs	Put two versions of your pipeline side by side -- same questions, different prompts or settings -- and see exactly where scores moved.
Prompt Hub	A shared library of reusable, versioned prompts, so your best prompt isn't buried in one notebook cell.

Pitfall

If a trace isn't appearing in the dashboard, the first thing to check is your `LANGCHAIN_TRACING_V2`, `LANGCHAIN_API_KEY`, and `LANGCHAIN_PROJECT` environment variables -- a missing or mismatched project name is the most common cause.

Remember

Project -> Trace -> Run is the vocabulary you'll use constantly once you open the LangSmith dashboard. Learn it once, and every screen in the tool makes sense.

INTERVIEW PREP

Q: How do you know a prompt change actually improved things?

A: Run both versions against the same dataset in LangSmith and compare the scores side by side -- don't just eyeball a few examples.

Q: What's the difference between a Trace and a Run in LangSmith?

A: A Trace is one full user request end to end; a Run is a single step inside that trace, such as one retrieval call or one LLM call. A trace is made up of one or more runs.

Q: How would you add tracing to a custom Python function that isn't part of a LangChain chain?

A: Wrap it with the `@traceable` decorator from the langsmith package -- it logs the function's inputs, outputs, and timing into the same trace as the surrounding LangChain calls.